WEEK 1

AIM: Extract data from different file formats and display the summary statistics.

Description: This program is designed to extract and analyze the contents of different file types. It uses the **os** module to identify file extensions, **pandas** for handling structured data formats like CSV, Excel, and JSON, and **PyMuPDF (fitz)** for extracting text from PDF files. Depending on the file type, the program either displays descriptive statistics (for CSV, Excel, JSON), prints extracted text with page and word counts (for PDFs), or shows raw text along with word counts (for TXT and Markdown). Unsupported file types are flagged with an appropriate message. The program runs in a loop, allowing users to repeatedly enter file paths until they type "exit". Error handling ensures that issues like missing files or reading errors are gracefully managed. This makes the script a flexible tool for preprocessing and analyzing different kinds of datasets and documents.

Code:

```
import os
import pandas as pd
import fitz  # PyMuPDF
def extract_file(file_path):
    try:
        ext = os.path.splitext(file_path)[1].lower()
        if ext == '.csv':
            df = pd.read_csv(file_path)
            print(df.describe(include='all'))
        elif ext in ['.xls', '.xlsx']:
            df = pd.read_excel(file_path)
            print(df.describe(include='all'))
```

```
        elif ext == '.json':
            df = pd.read_json(file_path)
            print(df.describe(include='all'))
        elif ext == '.pdf':
            doc = fitz.open(file_path)
            full_text = "".join(f"\n--- Page {i+1} ---\n{p.get_text()}" for i, p in
enumerate(doc))
            print(full_text)
            print(f"Total pages: {len(doc)}")
            print(f"Total words: {sum(len(p.get_text().split()) for p in doc)}")
        elif ext in ['.txt', '.md']:
            with open(file_path, 'r', encoding='utf-8') as f:
                text = f.read()
                print(text)
                print(f"Total words: {len(text.split())}")
        else:
            print(f"Unsupported file format: {file_path}")
    except Exception as e:
        print(f"Error processing file {file_path}: {e}")
def main():
    while True:
        path = input("Enter file path (or 'exit' to quit): ").strip()
        if path.lower() == 'exit':
            break
        if not os.path.isfile(path):
            print(f"File not found: {path}")
            continue
```

```
    extract_file(path)
if __name__ == "__main__":
  main()
```

Output:

```
PS D:\Sem 5\ML-2 lab> python -u "d:\Sem 5\ML-2 lab\A1.py"
Enter file path (or 'exit' to quit): D:\Sem 5\ML-2 lab\A1.txt
import os
import pandas as pd
import fitz  # PyMuPDF

def extract_file(file_path):
    try:
        ext = os.path.splitext(file_path)[1].lower()

        if ext == '.csv':
            df = pd.read_csv(file_path)
            print(df.describe(include='all'))


Total words: 24
Enter file path (or 'exit' to quit):
```

WEEK-2

AIM: Write a program that extracts the words (features) used in a sentence.

Description:

Library re → Used to find patterns in text (like words, numbers, punctuations).

sentence.lower() → Converts all letters in the sentence to lowercase.
Example: "Hi" becomes "hi".

re.findall(pattern, text) → Searches the text and returns all matches of the given pattern.

Pattern \b\w+\b

- \b → word boundary
- \w+ → one or more letters, digits, or underscore
- Used to extract words from text.

re.findall(r'[^\w\s]', sentence) → Used to find special characters in a sentence.

- \w → letters, digits, underscore
- \s → whitespace
- ^ inside brackets → means "except these"
- So this finds characters that are not words or spaces (special characters).

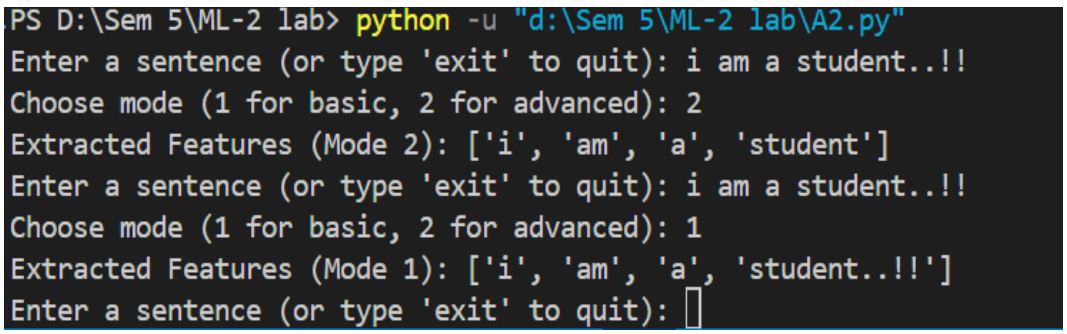
from_file(filename) → Function to take file input.

- Reads file contents
- Calls extract_features() on the text inside the file.

Code:

```
import re
def extract_features(sentence, mode=1):
    if mode == 1:
        return sentence.split()
    elif mode == 2:
        return re.findall(r"\b\w+\b", sentence)
    else:
        return []
def main():
    while True:
        sentence = input("Enter a sentence (or type 'exit' to quit): ")
        if sentence.lower() == 'exit':
            break
        mode = int(input("Choose mode (1 for basic, 2 for advanced): "))
        features = extract_features(sentence, mode)
        print(f"Extracted Features (Mode {mode}):", features)
if __name__ == "__main__":
    main()
```

Output:



WEEK-3

AIM: Write a program for edge detection to extract edge based features from a sample image.

Description:

1. Convert to Grayscale → Edges are based on intensity change, not color.

2. Gaussian Blur → Removes noise for better edge detection.

3. Canny Algorithm → Finds edges using gradients with two thresholds:

- threshold1 = lower bound
- threshold2 = upper bound

4. Output → Binary image showing white edges on black background.

5. Feature extraction → Number of edge pixels (cv2.countNonZero(edges)) can be used as a feature.

Code:

```
import cv2

import matplotlib.pyplot as plt

# Load image

image = cv2.imread('image.jpg')

gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

# Apply Gaussian blur (to reduce noise)

blurred = cv2.GaussianBlur(gray, (5, 5), 0)

# Apply Canny Edge Detector

edges = cv2.Canny(blurred, threshold1=100, threshold2=200)

# Display original and edges

plt.figure(figsize=(12,6))
```

```
plt.subplot(1,2,1)
plt.imshow(cv2.cvtColor(image, cv2.COLOR_BGR2RGB))
plt.title("Original Image")
plt.axis("off")
plt.subplot(1,2,2)
plt.imshow(edges, cmap='gray')
plt.title("Canny Edge Detection")
plt.axis("off")
plt.show()
# Print number of edge pixels
print("Number of edge pixels:", cv2.countNonZero(edges))
```

Output:



WEEK-4

Aim: Write a program to extract SURF/SIFT feature descriptors from a sample image.

Description:

SIFT – Scale Invariant Feature Transform

1. Detects keypoints in an image that are invariant to scale, rotation, illumination, affine transformations.

2. Uses Difference of Gaussian pyramid to find potential keypoints across scale.

3. Removes low contrast or poorly localized points.

4. Assigns a consistent orientation to each keypoint based on local image gradients.

5. 128-dimensional vector is created from orientation histograms (descriptor generation).

SURF – Speeded Up Robust Features

1.Uses integral images for fast filtering.

2.Uses Hessian matrix approximation for blob detection.

3.Uses a 64-dimensional descriptor for descriptor generation.

4.We need to use libraries like OpenCV in Python.

Code:

```
import cv2

import matplotlib.pyplot as plt

image = cv2.imread('image.jpg')

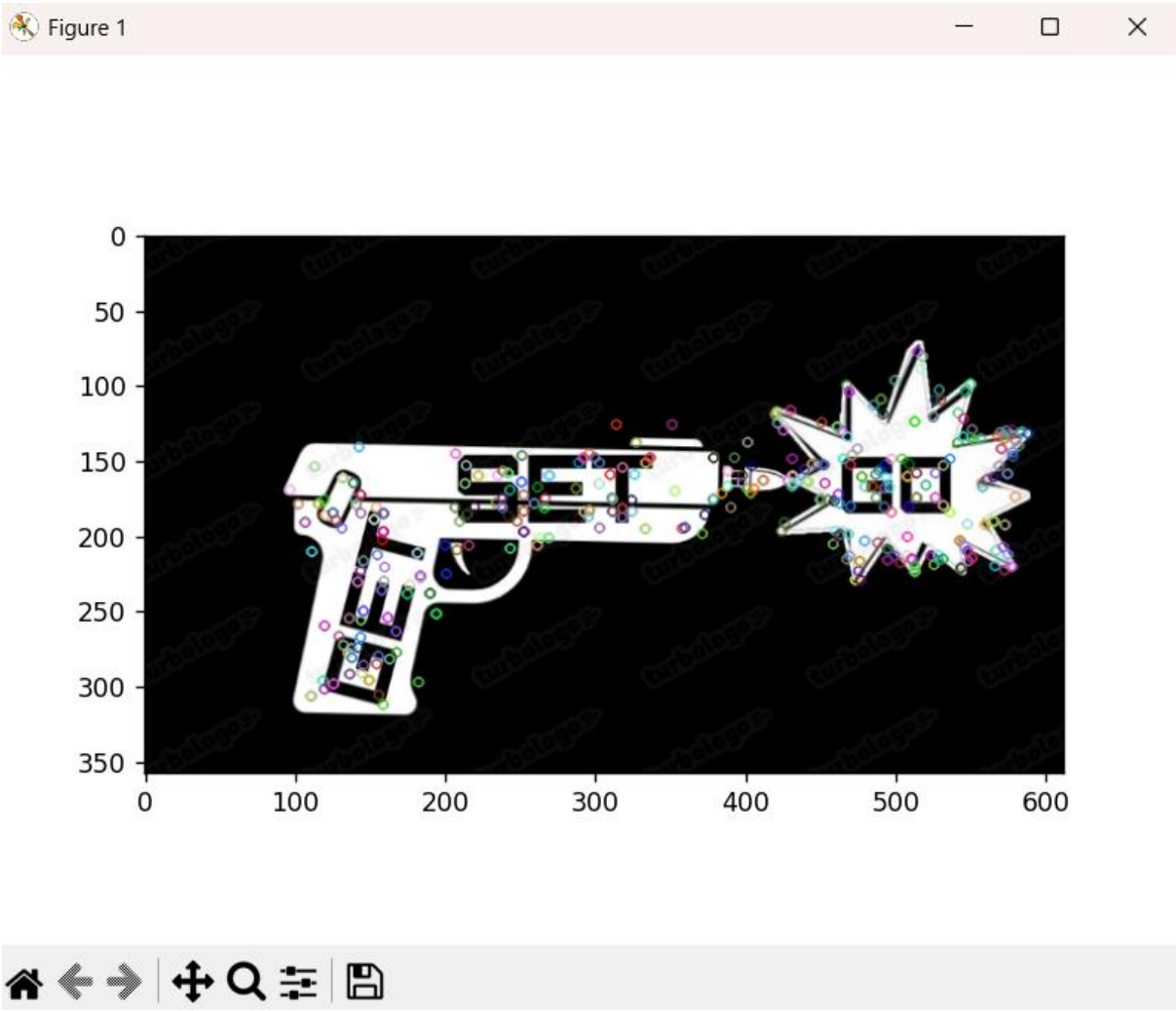
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

sift = cv2.SIFT_create()

keypoints, descriptors = sift.detectAndCompute(gray, None)
```

```
sift_image = cv2.drawKeypoints(gray, keypoints, None)
plt.imshow(sift_image)
plt.show()
```

Output:



WEEK-5

AIM: Write a program to perform Exploratory Data Analysis on real time datasets using the following approaches:

a) Univariate Analysis

b) Multivariate Analysis

c) Visualization using correlation matrix

Description:

a) Univariate Analysis

1. Analysis of a single variable at a time.
2. Understand the distribution, central tendency (mean, median, mode), spread (variance, standard deviation), and outliers of one variable.
3. Techniques:

   Histogram → shows frequency distribution.

   Boxplot → detects outliers and spread.

   Density plot → shows smooth distribution curve.
4. Example (Iris dataset):
   Checking distribution of petal length individually. We can see if it's normally distributed or skewed.

b) Multivariate Analysis

1. Analysis of two or more variables together to understand relationships.
2. Purpose: Discover patterns, associations, correlations, or class separations.
3. Techniques:

   Scatter plots → shows relation between two variables.

   Pair plots → shows relationships between all pairs.

   Grouped boxplots → compares distributions across categories.

4. Example (Iris dataset):
   Plotting petal length vs petal width shows that different Iris species separate clearly.

c) Correlation Matrix Visualization

1. Definition: A matrix that shows correlation coefficients (ranging from -1 to +1) between variables.
2. Purpose: Identify linear relationships (positive or negative) between features.
3. Techniques:

   Heatmap of correlation matrix → easy visualization.

   Values close to +1 → strong positive relation.

   Values close to -1 → strong negative relation.
4. Example (Iris dataset):
   Petal length and petal width have a high positive correlation, meaning as one increases, the other increases too.

Code:

```
# Exploratory Data Analysis (EDA) on Iris Dataset

import pandas as pd

import seaborn as sns

import matplotlib.pyplot as plt

from sklearn.datasets import load_iris

# 1. Load Dataset

iris = load_iris()

df = pd.DataFrame(data=iris.data, columns=iris.feature_names)

df['species'] = iris.target

# Map target numbers to actual species names

df['species'] = df['species'].map({0:'setosa', 1:'versicolor', 2:'virginica'})
```

```
print("First 5 Rows of Dataset:\n", df.head())
print("\nDataset Info:")
print(df.info())
print("\nSummary Statistics:")
print(df.describe())
# 5a. Univariate Analysis
print("\n--- Univariate Analysis ---")
# Distribution of each feature
df.hist(bins=20, figsize=(10, 8), edgecolor='black')
plt.suptitle("Univariate Analysis - Histograms")
plt.show()
# Boxplots to see outliers
plt.figure(figsize=(10,6))
sns.boxplot(data=df.drop(columns="species"))
plt.title("Boxplot of Features")
plt.show()
# 5b. Multivariate Analysis
print("\n--- Multivariate Analysis ---")
# Pairplot (scatterplot between all pairs)
sns.pairplot(df, hue="species", diag_kind="kde")
plt.suptitle("Multivariate Analysis - Pairplot", y=1.02)
plt.show()
# 5c. Correlation Matrix Visualization
print("\n--- Correlation Matrix ---")
corr = df.drop(columns="species").corr()
print(corr)
```

```
plt.figure(figsize=(8,6))
sns.heatmap(corr, annot=True, cmap="coolwarm", linewidths=0.5)
plt.title("Correlation Matrix Heatmap")
plt.show()
```

Output:

```
First 5 Rows of Dataset:
   sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm) species
0                5.1               3.5                1.4               0.2  setosa
1                4.9               3.0                1.4               0.2  setosa
2                4.7               3.2                1.3               0.2  setosa
3                4.6               3.1                1.5               0.2  setosa
4                5.0               3.6                1.4               0.2  setosa

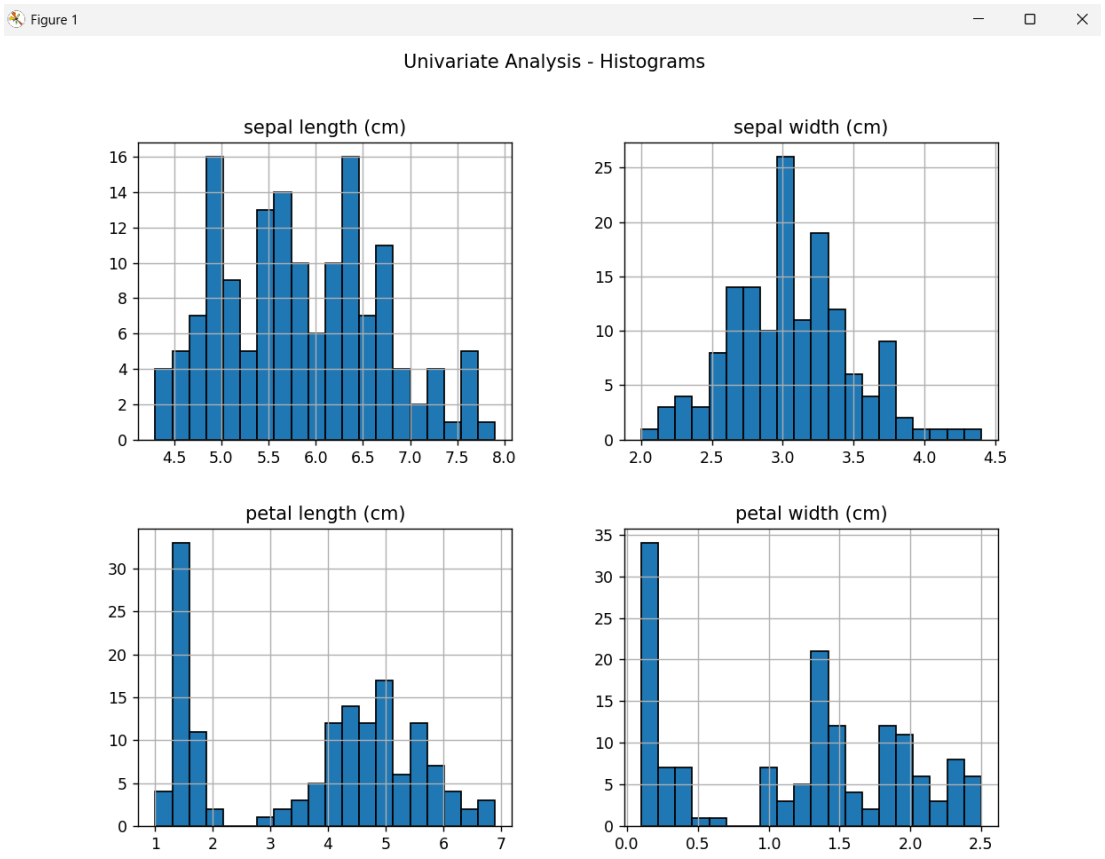
Dataset Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   sepal length (cm)  150 non-null    float64
 1   sepal width (cm)   150 non-null    float64
 2   petal length (cm)  150 non-null    float64
 3   petal width (cm)   150 non-null    float64
 4   species            150 non-null    object
dtypes: float64(4), object(1)
memory usage: 6.0+ KB
None

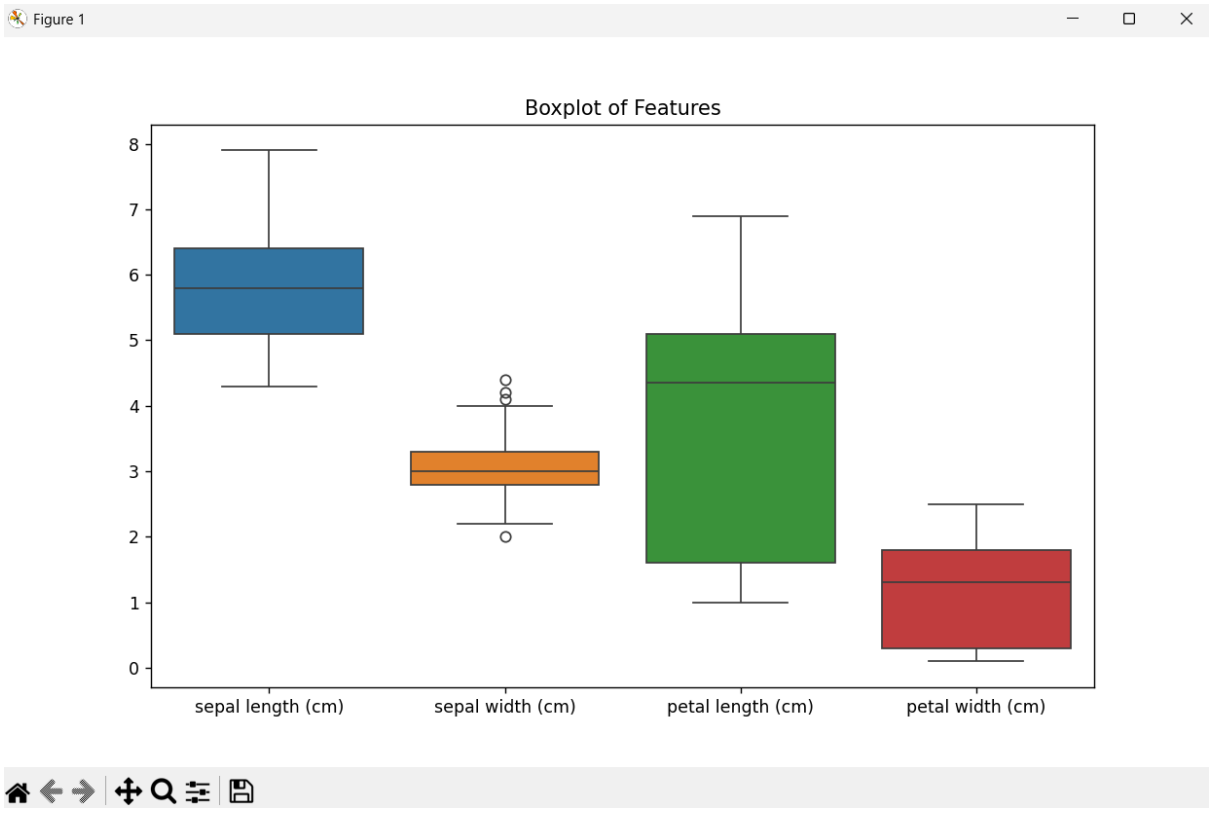
Summary Statistics:
       sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)
count         150.000000        150.000000         150.000000        150.000000
mean            5.843333          3.057333           3.758000          1.199333
std             0.828066          0.435866           1.765298          0.762238
min             4.300000          2.000000           1.000000          0.100000
25%             5.100000          2.800000           1.600000          0.300000
50%             5.800000          3.000000           4.350000          1.300000
75%             6.400000          3.300000           5.100000          1.800000
max             7.900000          4.400000           6.900000          2.500000
```

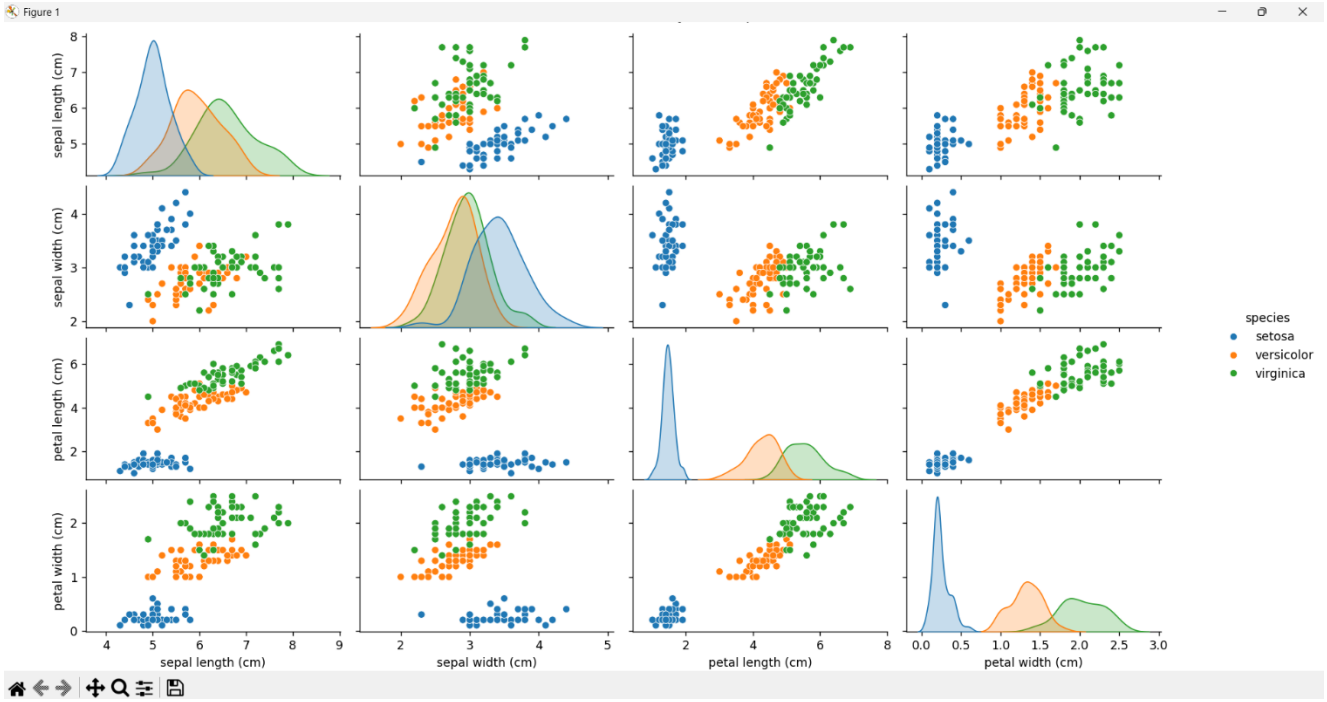
```
--- Univariate Analysis ---

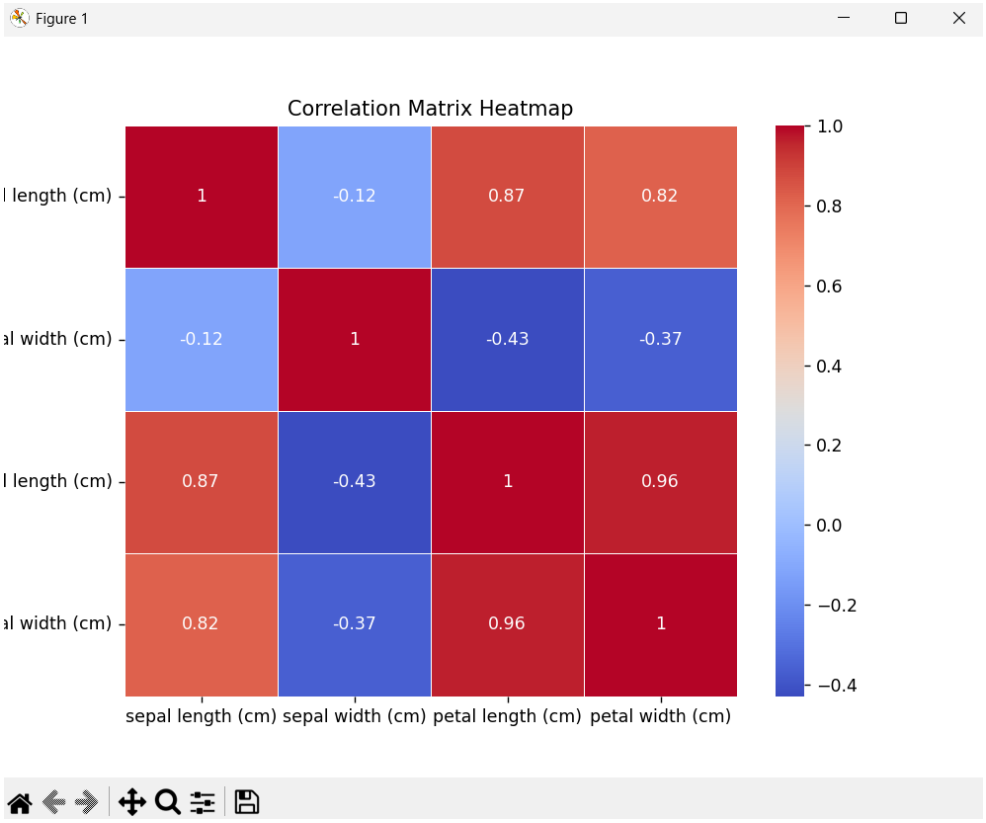
--- Multivariate Analysis ---

--- Correlation Matrix ---
                   sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)
sepal length (cm)           1.000000         -0.117570           0.871754          0.817941
sepal width (cm)           -0.117570          1.000000          -0.428440         -0.366126
petal length (cm)           0.871754         -0.428440           1.000000          0.962865
petal width (cm)            0.817941         -0.366126           0.962865          1.000000
```


Figure 1
Univariate Analysis - Histograms
sepal length (cm)
sepal width (cm)
petal length (cm)
petal width (cm)


Figure 1
Boxplot of Features
sepal length (cm)
sepal width (cm)
petal length (cm)
petal width (cm)


Figure 1
sepal length (cm)
sepal width (cm)
petal length (cm)
petal width (cm)
species
setosa
versicolor
virginica


Figure 1
Correlation Matrix Heatmap
1
-0.12
0.87
0.82
-0.12
1
-0.43
-0.37
0.87
-0.43
1
0.96
0.82
-0.37
0.96
1
sepal length (cm)
sepal width (cm)
petal length (cm)
petal width (cm)
1.0
0.8
0.6
0.4
0.2
0.0
−0.2
−0.4

WEEK-6

Aim: Write a program to perform any of the following Dimensionality Reduction techniques on real time datasets.

a) Principal Component Analysis

b) Single value Decomposition

c) Linear Discriminant Analysis

d) Factor Analysis

Description:

Principal Component Analysis (PCA) - Type: Unsupervised technique.

- Functionality:
    - Reduces dimensionality by finding new orthogonal axes (Principal Components).
    - First component captures maximum variance, second captures next, and so on.
    - Useful for data visualization and noise reduction.
- In program: We reduced 4 features of Iris to 2 PCs and plotted them.

Single Value Decomposition (SVD) - Type: Matrix Factorization method.

- Functionality:
    - Decomposes data matrix XXX into three matrices UΣVTU \Sigma V^TUΣVT.
    - Similar to PCA but works directly on the data matrix (without mean-centering).
    - Used in Latent Semantic Analysis (LSA), image compression, and recommender systems.
- In program: We used TruncatedSVD to reduce data to 2D and plotted.

Linear Discriminant Analysis (LDA) - Type: Supervised dimensionality reduction.

- Functionality:
    - Uses class labels to maximize separation between categories.
    - Projects data onto new axes that maximize between-class variance and minimize within-class variance.
    - Useful for classification and pattern recognition.
- In program: We reduced Iris dataset to 2 LD components → clear class separation.

Factor Analysis (FA) - Type: Statistical method.

- Functionality:
    - Assumes observed variables are influenced by a few latent (hidden) factors.
    - Reduces dimensionality by modeling correlations among variables.
    - Often used in psychology, social sciences, and survey analysis.
- In program: We extracted 2 hidden factors from the Iris dataset.

Code:

```
# Dimensionality Reduction Techniques Demo

import numpy as np

import pandas as pd

import matplotlib.pyplot as plt

from sklearn.datasets import load_iris

from sklearn.decomposition import PCA, TruncatedSVD, FactorAnalysis

from sklearn.discriminant_analysis import LinearDiscriminantAnalysis as LDA

# 1. Load Dataset

iris = load_iris()

X = iris.data      # features
```

```
y = iris.target    # labels
target_names = iris.target_names
print("Shape of Original Data:", X.shape)
# 6a. Principal Component Analysis (PCA)
pca = PCA(n_components=2)   # reduce to 2D
X_pca = pca.fit_transform(X)
print("\nPCA Explained Variance Ratio:", pca.explained_variance_ratio_)
# Plot PCA
plt.figure(figsize=(6,4))
for i, target in enumerate(np.unique(y)):
    plt.scatter(X_pca[y==target,0], X_pca[y==target,1],
label=target_names[target])
plt.title("PCA on Iris Dataset")
plt.xlabel("PC1")
plt.ylabel("PC2")
plt.legend()
plt.show()
# 6b. Single Value Decomposition (SVD) using TruncatedSVD
svd = TruncatedSVD(n_components=2)
X_svd = svd.fit_transform(X)
print("\nSVD Explained Variance Ratio:", svd.explained_variance_ratio_)
plt.figure(figsize=(6,4))
for i, target in enumerate(np.unique(y)):
    plt.scatter(X_svd[y==target,0], X_svd[y==target,1],
label=target_names[target])
plt.title("SVD on Iris Dataset")
plt.xlabel("SVD1")
```

```
plt.ylabel("SVD2")
plt.legend()
plt.show()
# 6c. Linear Discriminant Analysis (LDA)
lda = LDA(n_components=2)
X_lda = lda.fit_transform(X, y)
plt.figure(figsize=(6,4))
for i, target in enumerate(np.unique(y)):
    plt.scatter(X_lda[y==target,0], X_lda[y==target,1],
label=target_names[target])
plt.title("LDA on Iris Dataset")
plt.xlabel("LD1")
plt.ylabel("LD2")
plt.legend()
plt.show()
# 6d. Factor Analysis
fa = FactorAnalysis(n_components=2)
X_fa = fa.fit_transform(X)
plt.figure(figsize=(6,4))
plt.scatter(X_fa[:,0], X_fa[:,1], c=y, cmap="viridis", edgecolor="k")
plt.title("Factor Analysis on Iris Dataset")
plt.xlabel("Factor 1")
plt.ylabel("Factor 2")
plt.colorbar(label="Class")
plt.show()
print("\n--- Summary ---")
print("Original Shape:", X.shape)
```

```
print("After PCA Shape:", X_pca.shape)
print("After SVD Shape:", X_svd.shape)
print("After LDA Shape:", X_lda.shape)
print("After Factor Analysis Shape:", X_fa.shape)
```

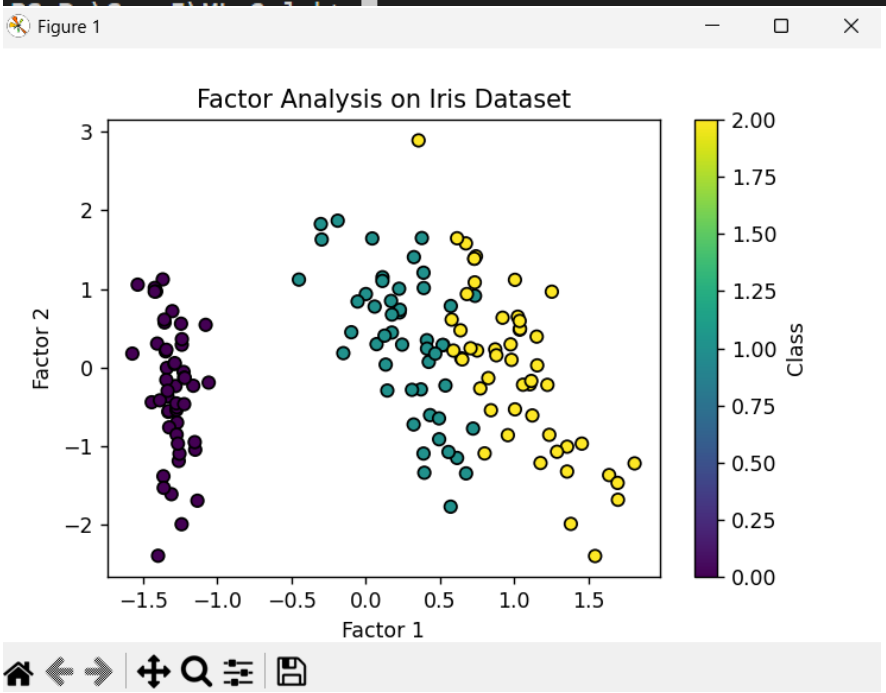
Output:

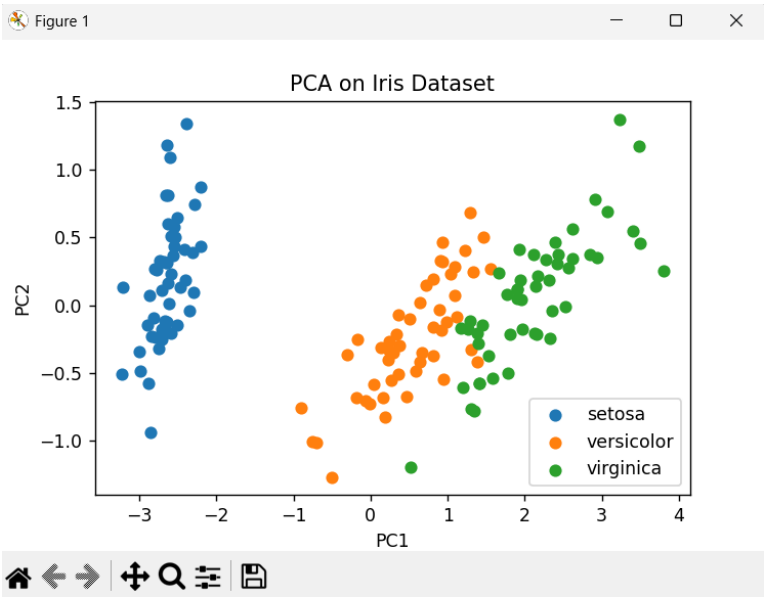
```
PS D:\Sem 5\ML-2 lab> python -u "d:\Sem 5\ML-2 lab\A6.py"
Shape of Original Data: (150, 4)

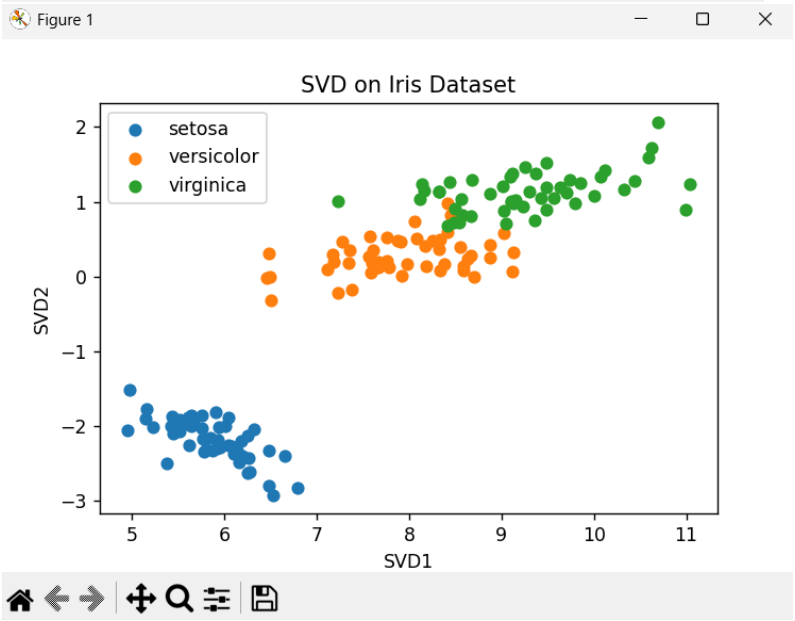
PCA Explained Variance Ratio: [0.92461872 0.05306648]

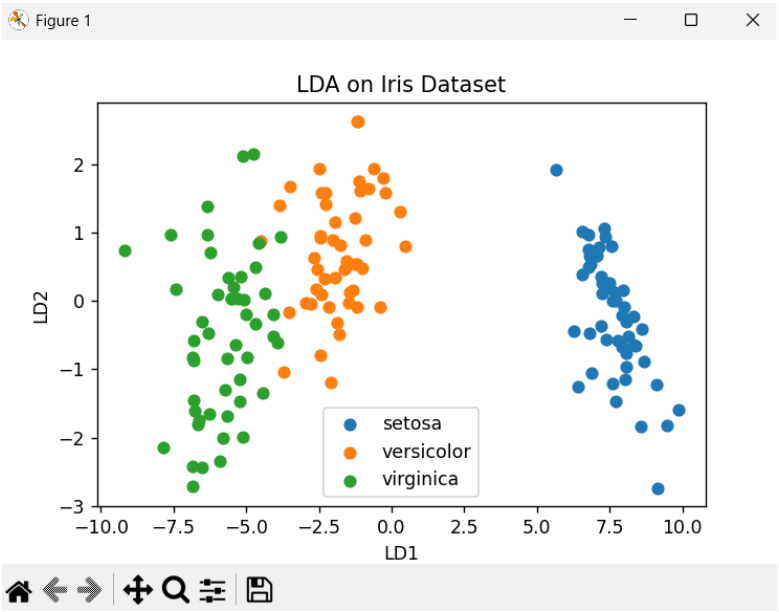
SVD Explained Variance Ratio: [0.52875361 0.44845576]

--- Summary ---
Original Shape: (150, 4)
After PCA Shape: (150, 2)
After SVD Shape: (150, 2)
After LDA Shape: (150, 2)
After Factor Analysis Shape: (150, 2)
```




Figure 1
PCA on Iris Dataset
PC2
PC1
setosa
versicolor
virginica


Figure 1
SVD on Iris Dataset
SVD2
SVD1
setosa
versicolor
virginica


Figure 1
LDA on Iris Dataset
LD2
LD1
setosa
versicolor
virginica